

Creating an Hyland Experience Application (Custom UI)

 Information last updated 22 sept 2025

SME	Jan Rusch
EPIC	CIC Custom User Interface and Custom Development

- 1  Instructions
 - 1.1 [How to Build Hyland Experience Application - Custom UI](#)
 - 1.2  [Complete Setup Process Flow](#)
 - 1.3  [Prerequisites](#)
 - 1.4  [Hyland Experience Workspace UI Template](#)
 - 1.4.1 [1.1 Create Your Custom UI in Studio Modeling](#)
 - 1.4.2 [1.2 Deploy and Configure in Studio Admin](#)
 - 1.4.3 [1.3 Configure Your Local Workspace](#)
 - 1.4.3.1 [1.3.1 Update the Configuration File](#)
 - 1.4.4 [1.4 Install Dependencies and Run](#)
 - 1.5 [What does npm ci do?](#)
 - 1.6 [Key differences from npm install:](#)
 - 1.6.1 [1.4.1 Environment Variable Setup Process](#)
 - 1.6.2 [1.4.2 What the Command Does Step-by-Step:](#)
 - 1.6.3 [1.4.3 Environment Variables Explained](#)
 - 1.6.4 [1.4.4 Why This Step is Critical](#)
 - 1.7  [Success! Your Workspace is Running](#)
 - 1.8  [Common Issues and Troubleshooting](#)
 - 1.8.1 [Content Services Navigation Issues](#)
 - 1.8.2 [Port Already in Use](#)
 - 1.9  [Next Steps](#)
 - 1.10  [Best Practices](#)
- 2  [Related articles](#)

Instructions

How to Build Hyland Experience Application - Custom UI

Hey there! Today, I'm going to walk you through the complete process of setting up a Custom UI from the Hyland Experience Workspace template.

This is your foundation for building powerful, customized applications on the Hyland Experience platform.

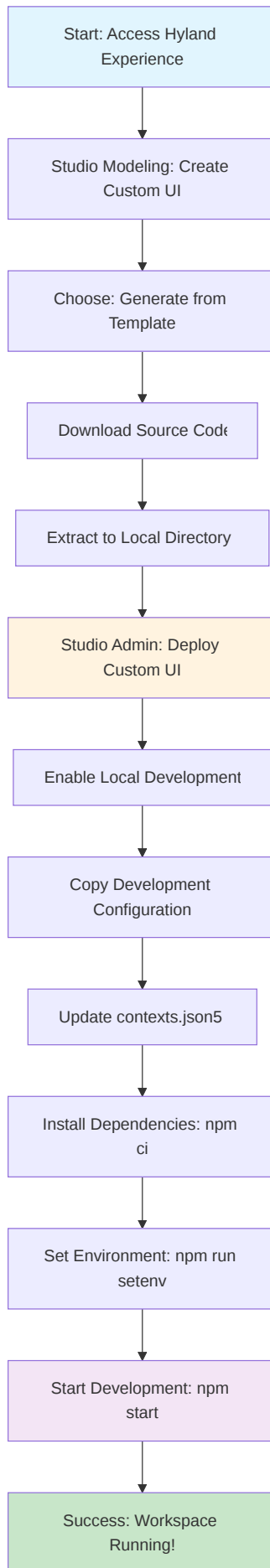
What you'll learn today:

- How to create and download a Custom UI from Studio Modeling
- How to configure your local development environment
- How to get your workspace running locally
- Common troubleshooting tips

Let's dive right in!



Complete Setup Process Flow



Prerequisites

Before we start, make sure you have:

- **Node.js 18 or higher** (the workspace requires Node 18-22)
- **npm** (comes with Node.js)
- **Access to Hyland Experience Studio Modeling**
- **Access to Hyland Experience Studio Admin**
- **A code editor** (VS Code recommended)
- **Git** for version control (highly recommended)

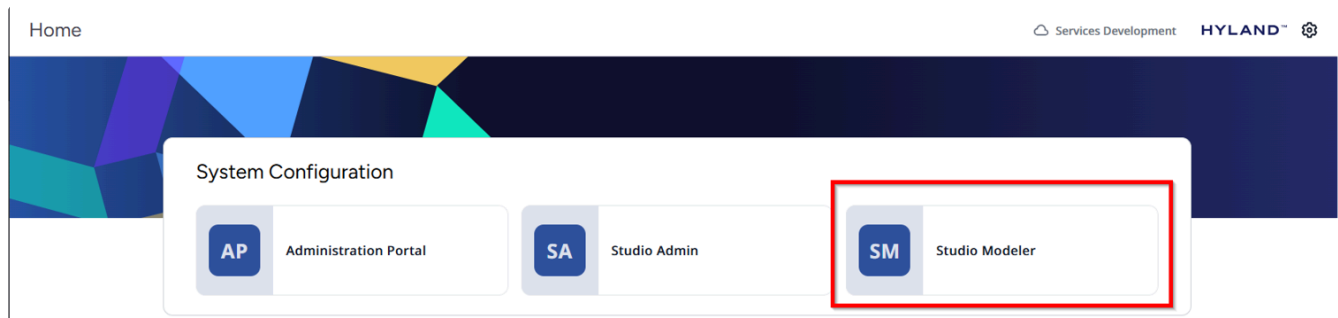
Hyland Experience Workspace UI Template

The Workspace approach is perfect for most developers who want to quickly build custom UIs with standard Hyland Experience features.

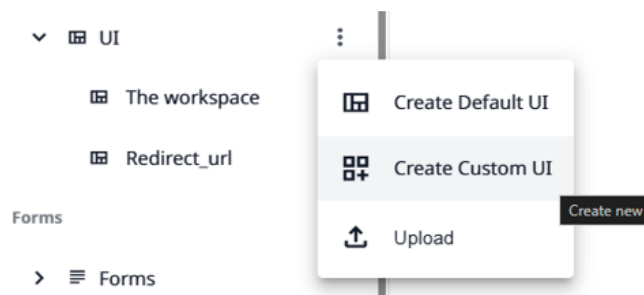
1.1 Create Your Custom UI in Studio Modeling

First, let's create your Custom UI in Studio Modeling:

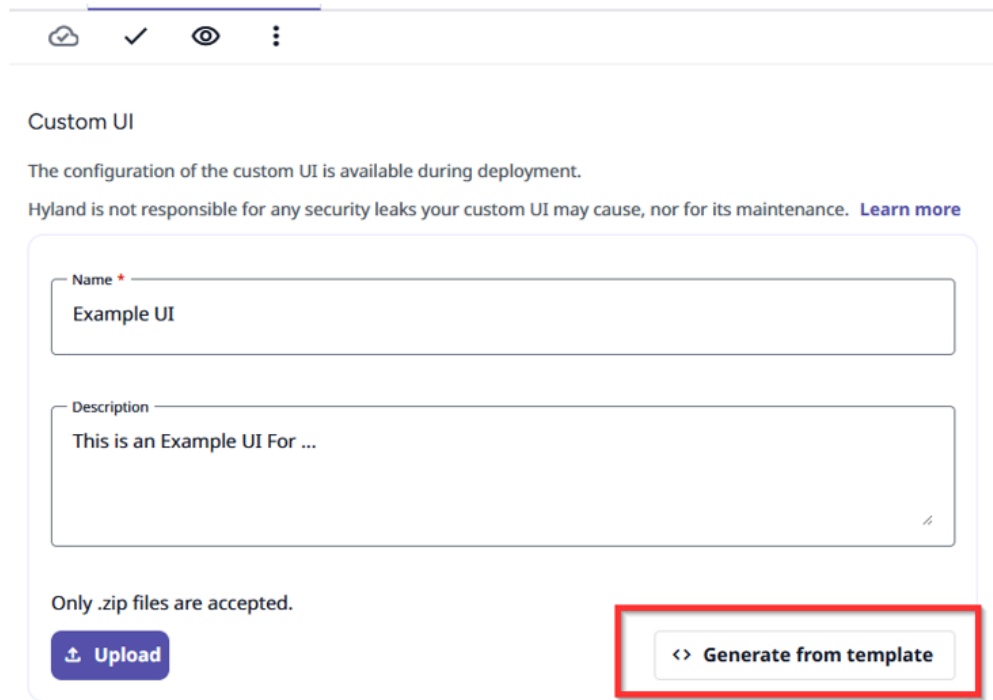
1. **Navigate to Studio Modeling** and log in to your Hyland Experience environment



2. **Create a new Custom UI** by clicking the "More" button



3. **Choose "Generate from template"** - this gives you the latest Hyland Experience Workspace template



Custom UI

The configuration of the custom UI is available during deployment.

Hyland is not responsible for any security leaks your custom UI may cause, nor for its maintenance. [Learn more](#)

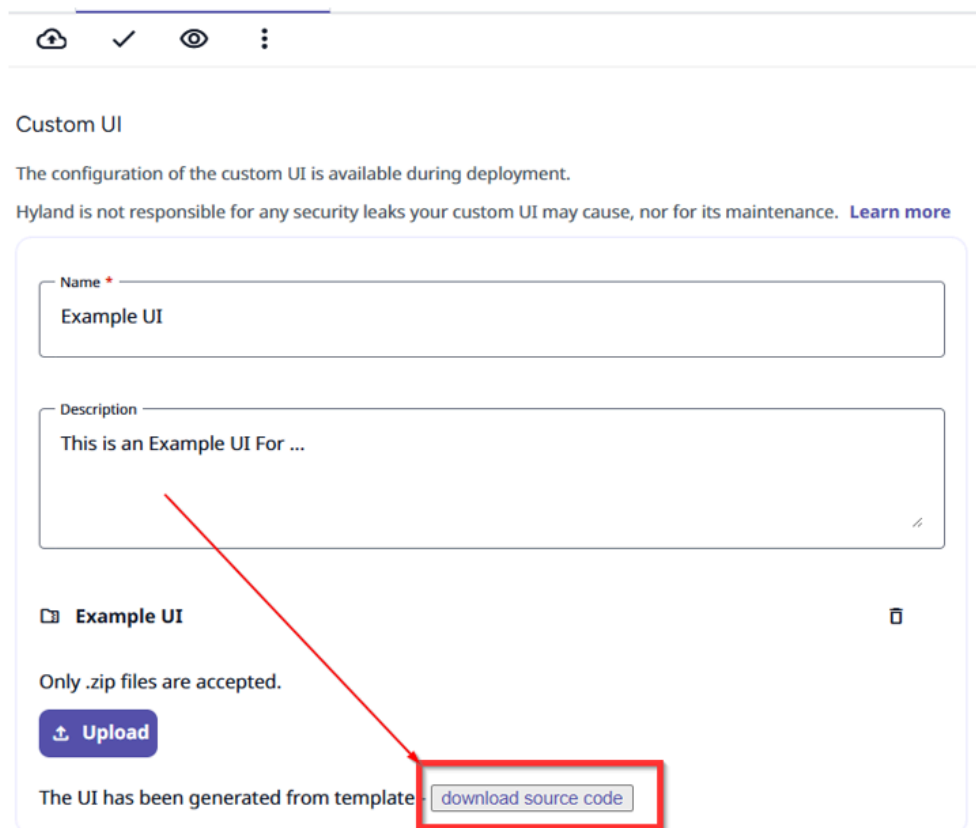
Name *
Example UI

Description
This is an Example UI For ...

Only .zip files are accepted.

[Upload](#) [<> Generate from template](#)

4. **Download the source code** - once generated, you'll see a "Download Source Code" option



Custom UI

The configuration of the custom UI is available during deployment.

Hyland is not responsible for any security leaks your custom UI may cause, nor for its maintenance. [Learn more](#)

Name *
Example UI

Description
This is an Example UI For ...

[Example UI](#)

Only .zip files are accepted.

[Upload](#)

The UI has been generated from template [download source code](#)

5. Extract the zip file to your local development directory

Name	Date modified	Type	Size
.private-packages	22.09.2025 11:17	File folder	
apps	22.09.2025 11:17	File folder	
config	22.09.2025 11:17	File folder	
developer-docs	22.09.2025 11:17	File folder	
e2es	22.09.2025 11:17	File folder	
libs	22.09.2025 11:18	File folder	
scripts	22.09.2025 11:18	File folder	
tools	22.09.2025 11:18	File folder	
.env	18.09.2025 10:43	ENV File	1 KB
.gitignore	18.09.2025 10:33	GITIGNORE File	2 KB
.nvmrc	18.09.2025 10:33	NVMRC File	1 KB
.nxignore	18.09.2025 10:33	NXIGNORE File	1 KB
jest.config.ts	18.09.2025 10:33	TypeScript Source ...	1 KB
jest.preset.js	18.09.2025 10:33	JSFile	1 KB
karma.conf.js	18.09.2025 10:33	JSFile	2 KB
license-header.txt	18.09.2025 10:33	Textdokument	1 KB
nx.json	18.09.2025 10:43	JSON File	5 KB
package.json	18.09.2025 10:43	JSON File	9 KB
package-lock.json	18.09.2025 10:43	JSON File	1.806 KB
proxy-helpers.js	18.09.2025 10:33	JSFile	9 KB
README.md	18.09.2025 10:33	Markdown Source...	3 KB
tsconfig.adf.json	18.09.2025 10:43	JSON File	7 KB
tsconfig.base.json	18.09.2025 10:43	JSON File	8 KB
tsconfig.json	18.09.2025 10:33	JSON File	1 KB

6. Save your Custom UI in Studio Modeling

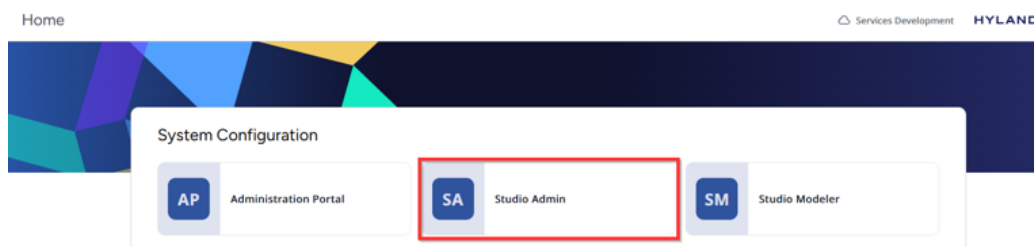
7. Release the project to make it available for deployment

i Give your Custom UI a descriptive name that reflects your project's purpose!

1.2 Deploy and Configure in Studio Admin

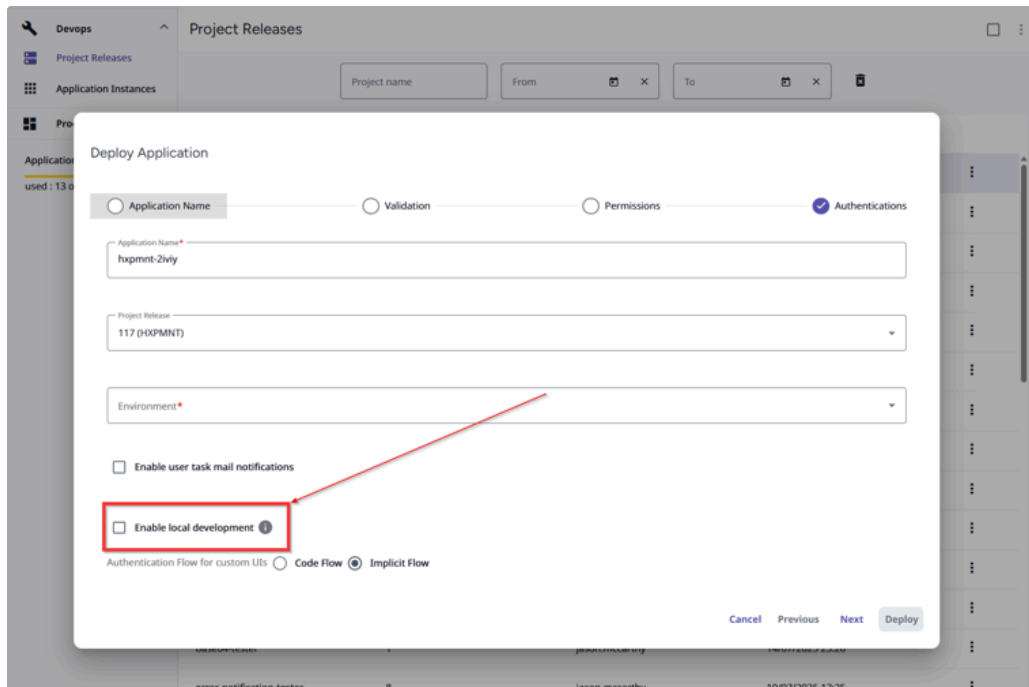
Now let's get your Custom UI deployed and configured:

1. Go to Studio Admin and navigate to your project

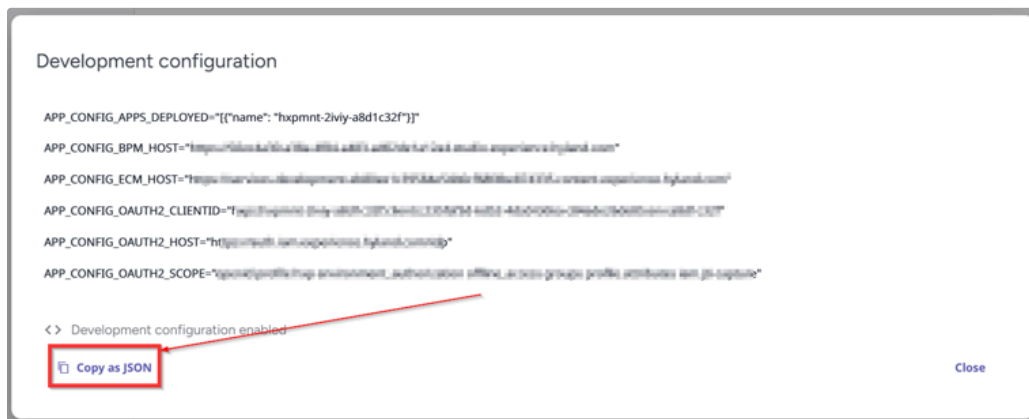


2. Deploy the project with your newly created Custom UI

⚠ IMPORTANT: Check the "Enable Local Development" checkbox during deployment



3. **Access Development Configuration** from the running Application Instance menu
4. **Copy the JSON configuration** - this contains all the connection details you'll need



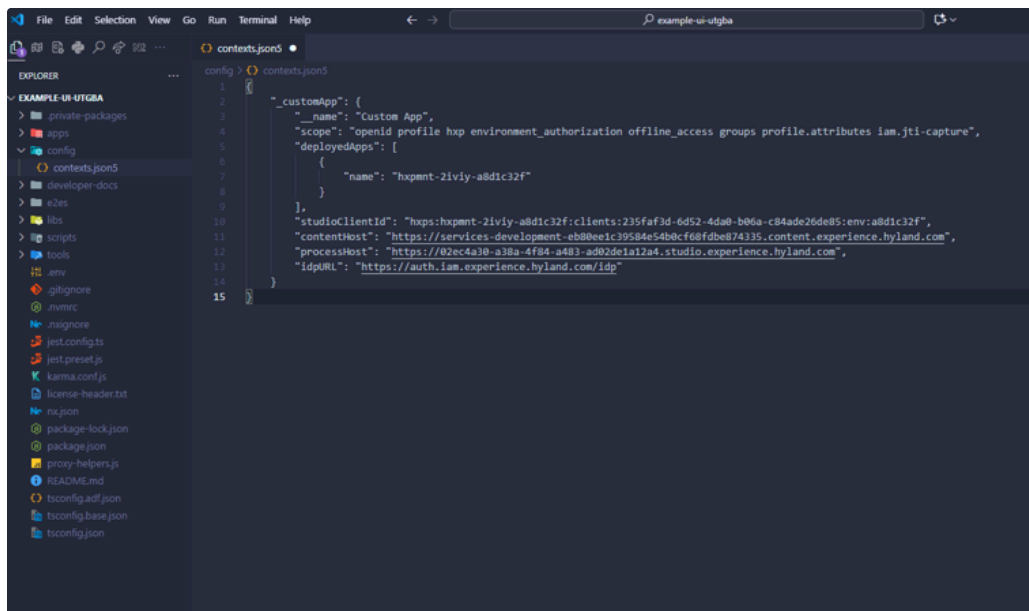
i This JSON is your golden ticket 🎟️ - it contains all the API endpoints, authentication settings, and environment configurations your local development needs.

1.3 Configure Your Local Workspace

Now let's set up your local development environment:

1.3.1 Update the Configuration File

1. **Navigate to your extracted workspace folder**
2. **Open** `config/contexts.json5` in your favorite code editor
3. **Replace the entire contents** with the JSON you copied from Studio Admin



4. Save the file

1.4 Install Dependencies and Run

Now for the moment of truth - let's get your workspace running:

```
1 # Install dependencies
2 npm ci
```

What is npm ci ?

`npm ci` (short for "clean install") is a command designed for automated environments like continuous integration (CI) pipelines, production deployments, and any scenario where you need a reliable, reproducible build process.

What does `npm ci` do?

1. **Deletes `node_modules` folder:** It completely removes the existing `node_modules` directory to ensure a clean state
1. **Uses `package-lock.json`:** It installs dependencies based on the exact versions specified in `package-lock.json` (or `npm-shrinkwrap.json`)
1. **Faster installation:** It's typically faster than `npm install` because it skips certain user-oriented features
1. **Fails on version conflicts:** If `package.json` and `package-lock.json` are out of sync, the command will fail rather than trying to resolve conflicts

Key differences from `npm install`:

Feature	<code>npm install</code>	<code>npm ci</code>
Purpose	Development workflow	Automated/production environments
Speed	Slower (resolves dependencies)	Faster (uses lock file)
Lock file	Updates <code>package-lock.json</code>	Requires existing <code>package-lock.json</code>
Conflict resolution	Attempts to resolve conflicts	Fails on conflicts
Clean install	No (incremental)	Yes (always clean)
User features	Includes user configs, etc.	Skips user-specific features

```
✓ 2/2 dependent project tasks succeeded [0 read from cache]

Hint: you can run the command with --verbose to see the full dependent project outputs

> nx run monorepo-builders:build

Compiling TypeScript files for project "monorepo-builders"...
Done compiling TypeScript files for project "monorepo-builders".

NX Successfully ran target build for project monorepo-builders and 2 tasks it depends on (8s)

added 3618 packages, and audited 3619 packages in 6m

510 packages are looking for funding
  run `npm fund` for details

16 vulnerabilities (4 low, 4 moderate, 1 high, 7 critical)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

```
1 # Set up environment
2 npm run setenv
```

```
Environment setup
App specific variables written into
apps/workspace-hxp/.env

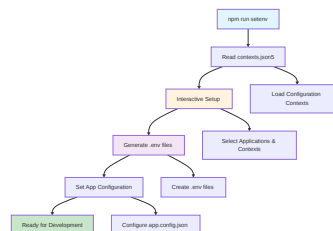
Nice job, you are all set!

PS C:\Users\jrusch\Downloads\example-ui-utgba> npm run setenv
```

Deep Dive: What npm run setenv Actually Does

The `npm run setenv` command is a crucial step that sets up your local development environment. Here's exactly what it does:

1.4.1 Environment Variable Setup Process



1.4.2 What the Command Does Step-by-Step:

- Reads Configuration:** Loads `config/contexts.json5` to get available environment contexts
- Interactive Setup:** Prompts you to:
 - Select which applications to configure (e.g., `workspace-hxp`)
- Generates .env Files:** Creates environment files with variables like:

```
1 # Application variables
2 APP_CONFIG_ECM_HOST=<https://the-hyland-experience-host.com>
3 APP_CONFIG_BPM_HOST=<https://the-hyland-experience-host.com>
4 APP_CONFIG_OAUTH2_HOST=<https://the-hyland-experience-host.com>
5 APP_CONFIG_OAUTH2_CLIENTID=your-client-id
6 APP_CONFIG_OAUTH2_SCOPE=openid profile email
7 NUCLEUS_API_HOST=<https://the-hyland-experience-host.com>
8 # ... and many more
```

- Configures App Settings:** Sets up the application configuration that will be used at runtime

1.4.3 Environment Variables Explained

The `setenv` command sets up these key categories of variables:

OAuth2 Authentication:

- `APP_CONFIG_OAUTH2_HOST` : TheHyland Experience host URL
- `APP_CONFIG_OAUTH2_CLIENTID` : OAuth2 client ID from Studio Admin
- `APP_CONFIG_OAUTH2_SCOPE` : OAuth2 scopes for authentication

API Endpoints:

- APP_CONFIG_ECM_HOST : Content Services API endpoint
- APP_CONFIG_BPM_HOST : Process Services API endpoint
- NUCLEUS_API_HOST : Nucleus API endpoint

Application Configuration:

- APP_CONFIG_PROVIDER : Provider configuration
- APP_CONFIG_SERVER_TIME_URL : Server time synchronization
- UI_TYPE : UI type configuration

Customization:

- APP_CONFIG_CUSTOM_HEADER_LOGO : Custom header logo path
- APP_CONFIG_CUSTOM_HEADER_COLOR : Header color customization
- APP_CONFIG_CUSTOM_CSS_PATH : Custom CSS file path

1.4.4 Why This Step is Critical

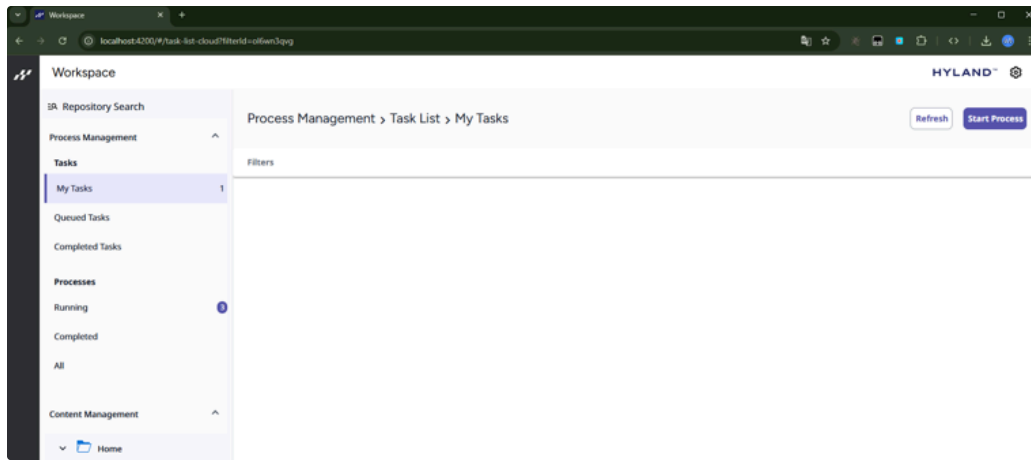
Without `npm run setenv`, your workspace would:

- ❌ Not know how to connect to your Hyland Experience environment
- ❌ Have no authentication configuration
- ❌ Use default/empty values for all API endpoints
- ❌ Fail to start or run with errors

The command essentially "tells" your local workspace how to connect to your specific Hyland Experience environment by setting all the necessary environment variables that the application needs to function.

```
1 # Start development server
2 npm start workspace-hxp
```

```
Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
Deployment
Fetch app: hxpmt-2iviy-a8d1c32f
Target for /hxpmt-2iviy-a8d1c32f: https://02ec4a30-a38a-4f84-a483-ad02de1a12a4.studio.experience.hyland.com
Target for /identity-adapter-service https://02ec4a30-a38a-4f84-a483-ad02de1a12a4.studio.experience.hyland.com
(!) invalid "base" option: ".". The value can only be an absolute URL, ".*", or an empty string.
  → Local: http://localhost:4200/
  → Network: http://172.16.0.6:4200/
  → Network: http://192.168.75.1:4200/
  → Network: http://192.168.40.1:4200/
  → Network: http://172.29.224.1:4200/
  → press h + enter to show help
□
```



Success! Your Workspace is Running

If everything went smoothly, you should see:

- Your browser automatically opens to `<http://localhost:4200>`
- The Hyland Experience Workspace interface loads
- You can navigate through the different sections
- All standard Hyland Experience features are available

Common Issues and Troubleshooting

Content Services Navigation Issues

Problem: Content Services section shows "No content available".

Why it happens: CORS preflight checks in Chrome can fail for Content Services API calls during local development.

Workaround: This is a known limitation of local development. The full functionality will work when deployed to your Hyland Experience environment.

Port Already in Use

Problem: Port 4200 is already in use.

Solution: Stop the other service using port 4200.

 Do not change the port of the workspace, otherwise you will no longer be able to log in to the identity server.

Next Steps

Now that your workspace is running, you're ready to:

- **Create custom plugins** (check out our "Creating a Plugin Page" tutorial)
 - **Build form widgets** (see "Creating Custom Forms Widget" tutorial)
 - **Deploy your changes** (see "Packaging a Custom UI" tutorial)
-

Best Practices

- **Use the plugin system** for new features
 - **Follow Angular best practices** for component development
 - **Test locally** before deploying
 - **Keep your workspace updated** with the latest template
 - **Use TypeScript** for better development experience
 - **Implement proper error handling** for API calls
-

Thanks happy coding! 

Related articles